

1. ARQUITETURA EM CAMADAS (TUS-2370)

Camada	Classes principais
Model	AcademicClass, Assessment (abs), Exam, Assignment, PracticalAssignment, Seminar, User (abs), Administrator, Professor, Role, PersistenceType
Repository	AcademicClassRepository (interface), InMemoryAcademicClassRepository, TxtAcademicClassRepository, JsonAcademicClassRepository, XmlAcademicClassRepository, TxtUserRepository
Service	AcademicService (fachada), ClassService, AssessmentService, PersistenceService, ReportService
Controller	AcademicSystemController, AuthenticationController
View	ConsoleView (CLI), JavaFX (GUI — colega)
Security	AuthenticationService, AuthorizationService
Logging	SecurityAuditLogger, PersistenceAuditLogger, ReportAuditLogger (SLF4J/Logback)
Validation	DomainValidator — Jakarta Bean Validation (@NotBlank, @Pattern, @DecimalMin/@Max)
Exception	AcademicSystemException, KeyboardInputException e 8 subclasses específicas

2. SUAS USER STORIES

US / TUS	O que faz / onde está no código
US-2361 Registrar avaliações	AssessmentService.registerAssessment() — valida código da turma, instancia o tipo correto via switch, valida com DomainValidator, adiciona à turma
TUS-2362 Persistir em TXT	TxtAcademicClassRepository.persistAll() — escreve cada turma e avaliação em arquivo .txt; chama PersistenceAuditLogger ao final
US-2363 Cadastrar turmas	ClassService.registerClass() — cria AcademicClass, valida @Pattern(CS101), salva no repositório
US-2364 Menu CLI	ConsoleView.start() — loop login → menu diferente por Role (ADMIN: 9 opções, PROFESSOR: 5); toda lógica delegada ao controller
TUS-2365 Lombok	@Getter @Setter @EqualsAndHashCode @ToString em AcademicClass e Assessment; User sobrescreve toString() para ocultar senha
US-2366 Autenticação	AuthenticationService.authenticate() — busca em TxtUserRepository, compara senha, grava log. requireRole() bloqueia operações por perfil. Senha NUNCA é logada
US-2367 Exceções domínio	Hierarquia: AcademicSystemException → ClassNotFoundException, InvalidAssessmentException, InvalidClassException. Capturadas no ConsoleView separado das de input
US-2368 Exceções teclado	Hierarquia: KeyboardInputException → InvalidMenuOptionException, InvalidNumericInputException. Lançadas nos métodos parseInt/parseDouble da ConsoleView
US-2369 Exceções segurança	SecurityException → AuthenticationException (credenciais erradas), AuthorizationException (perfil insuficiente). Log via SecurityAuditLogger antes de lançar
TUS-2370 Controller	AcademicSystemController desacopla View dos Services. ConsoleView nunca acessa repositório diretamente. Cada método verifica role antes de agir
TUS-2371 Bean Validation	DomainValidator.validate(obj) — usa ValidatorFactory estático, agrega todas as violações em uma única AcademicSystemException com mensagem clara
US-2372 Tipo de persistência	PersistenceService mantém PersistenceType (TXT/XML/JSON); buildRepository() instancia o repositório correto; só ADMIN pode alterar via controller

3. PERGUNTAS PROVÁVEIS DO PROFESSOR

P: Por que User é abstrata?	R: Nenhum usuário existe sem perfil concreto. Administrator e Professor têm construtores que fixam o Role, impedindo criação de User genérico.
P: Por que Assessment é abstrata?	R: Cada tipo (Exam, Seminar...) define getType(). Não faz sentido instanciar uma 'avaliação sem tipo'. O polimorfismo permite tratar todos uniformemente.
P: O que é o padrão Delegate nos repositórios?	R: Txt/Xml/JsonRepository delegam save/find ao InMemoryRepository (memória = fonte de verdade em runtime) e só implementam persistAll() para gravação em arquivo.
P: Por que existem ClassService E AssessmentService separados?	R: Princípio da Responsabilidade Única (SRP). ClassService só cuida de turmas; AssessmentService só de avaliações. AcademicService é a fachada que os une.
P: Como funciona o controle de acesso?	R: authService.requireRole(Role.ADMIN) no controller lança AuthorizationException antes de executar qualquer operação restrita. A View captura e exibe [ACCESS DENIED].
P: Como a senha é protegida?	R: toString() de User é sobrescrito manualmente (mesmo com Lombok) para nunca incluir o campo password. Os audit loggers recebem só o username.
P: O que é o DomainValidator?	R: Centraliza a validação Jakarta BV: tem um Validator estático inicializado no bloco static{}. validate(obj) agrega todas as violações e lança AcademicSystemException.
P: Por que duas hierarquias de exceção?	R: Erros de domínio (AcademicSystemException) e erros de teclado (KeyboardInputException) têm causas e tratamentos diferentes. O Main captura cada uma separadamente.
P: Como trocar de TXT para JSON sem reiniciar?	R: PersistenceService.setPersistenceType(JSON); buildRepository() usa switch expression de Java 21 para retornar o repositório correto. O estado em memória não muda.
P: O que o ConsoleView NÃO faz?	R: Não contém lógica de negócio nem acessa repositório diretamente. Só lê Scanner, chama controller, e exibe resultado — separação total de responsabilidades.
P: Para que serve AuthenticationController?	R: Desacopla a GUI JavaFX (colega) do AuthenticationService. A tela de login chama AuthenticationController.authenticate(), não o service diretamente.
P: O que é SLF4J e por que usá-lo?	R: API de logging agnóstica de implementação. O código usa LoggerFactory.getLogger(); o Logback (configurado no logback.xml) faz a gravação real. Fácil trocar de impl.
P: O que @EqualsAndHashCode(of="code") faz?	R: Gera equals e hashCode baseados APENAS no campo code. Duas AcademicClass com mesmo código são consideradas iguais, independente do título ou lista de avaliações.
P: Como rodar o projeto?	R: mvn compile → mvn test → mvn package → java -jar target/academic-system-1.0-SNAPSHOT.jar (CLI) ou mvn javafx:run (GUI). Credenciais: admin/admin123.

4. TRECHOS-CHAVE PARA CITAR

// US-2366 – senha nunca logada @Override public String toString() { return "User{username='"+ username +"', role='"+ role +"}"; } // US-2372 – switch expression Java 21 return switch(currentType) { case TXT -> new TxtAcademicClassRepository(mem); case XML -> new XmlAcademicClassRepository(mem); case JSON -> new JsonAcademicClassRepository(mem); }; // TUS-2365 – Lombok no modelo @Getter @Setter @EqualsAndHashCode(of="code") @ToString public class AcademicClass { ... }	// TUS-2371 – validação centralizada public static <T> void validate(T obj) { var v = VALIDATOR.validate(obj); if(!v.isEmpty()) throw new AcademicSystemException("Validation: "+...); } // US-2369 – requireRole public void requireRole(Role r) { if(currentUser==null currentUser.getRole()!=r){ SecurityAuditLogger.authorizationFailure(...); throw new AuthorizationException(...); } } // US-2367/2368 – captura separada na View catch(InvalidMenuOptionException e){ ... } catch(AcademicSystemException e){ ... } catch(AuthorizationException e){ ... }
---	--